



Tecnológico de Monterrey

Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Gabriela Michelle Lagos Aguilar - A01648192

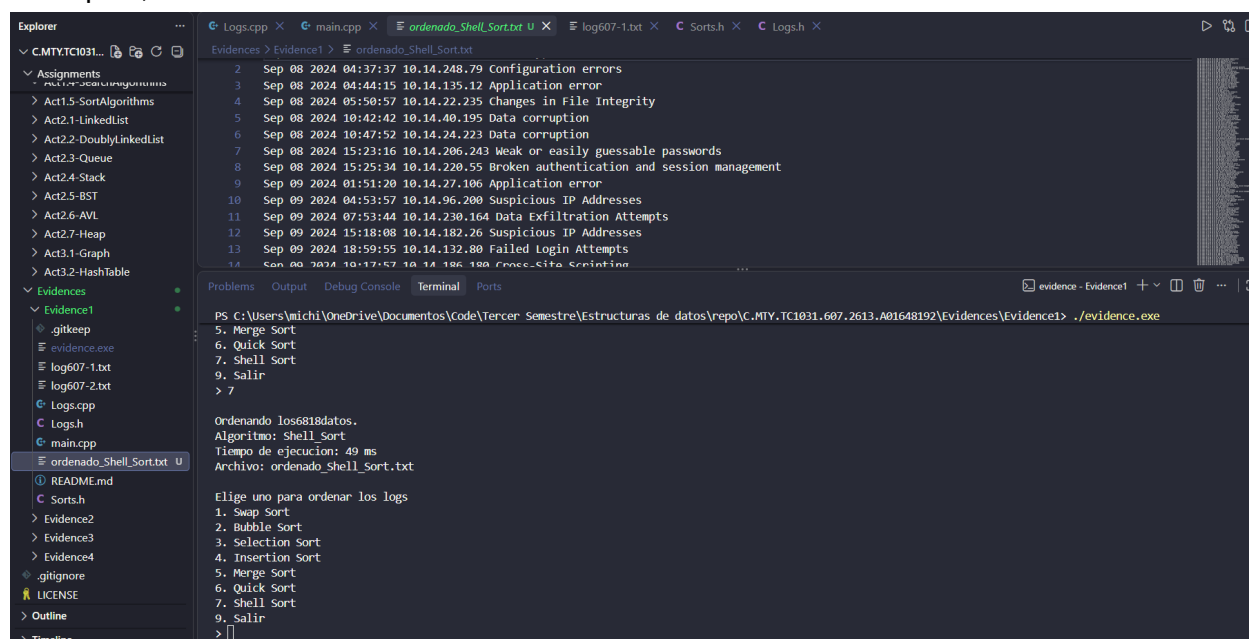
20/09/2026

Plan de pruebas

Para el código decidí compartimentar el proceso, asegurando que cada fase funcionara bien y hacer el push. Lo que me preocupaba era el tratamiento seguro de los datos para evitar problemas en mi computadora. Y por temas logísticos también figuré que iba a ser mucho más fácil cerebralmente hablando hacer el código poquito por poquito para no abrumarme y además tener registro en los commits de las partes iniciales funcionando perfectamente por si requería revisar mis versiones previas.

Prueba

Prueba 1, solo el menú de elección para ordenar el archivo de log607-1.txt. También crea, como pide, un archivo con los datos ordenados.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project structure with folders like 'Assignments' and 'Evidences'. The 'Evidences' folder is expanded, showing files like 'Evidence1', 'Evidence2', etc. The main editor window displays a log file with entries such as 'Configuration errors', 'Application error', 'Changes in File Integrity', 'Data corruption', 'Weak or easily guessable passwords', 'Broken authentication and session management', 'Suspicious IP Addresses', 'Data Exfiltration Attempts', and 'Failed Login Attempts'. The bottom terminal window shows the execution of a program named 'evidence.exe'. The program prompts the user to select a sorting algorithm from a list: 1. Swap Sort, 2. Bubble Sort, 3. Selection Sort, 4. Insertion Sort, 5. Merge Sort, 6. Quick Sort, 7. Shell Sort, 9. Salir. The user has selected '7' (Shell Sort). The program then displays the sorted data and the execution time.

Aún faltaba poder elegir incluir el sorting del segundo archivo e incluir las otras opciones en el menú, pero primero quería tener la base del Sorting y la creación de un nuevo archivo, que no había hecho antes. Arriba se ve la primera prueba que hice con el código.

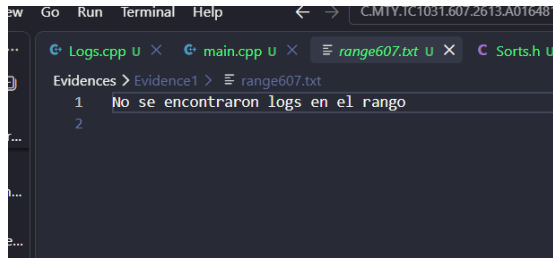
La segunda Prueba Incluía la segunda base de datos y el diálogo de predicciones. Leí las instrucciones y me di cuenta que se especificaba el nombre del archivo, entonces también agregué eso.

Finalmente, agregué el código de los rangos, aunque primero lo hice con un linear search, solo para ver si corría bien el código.

Desafortunadamente, no tomé capturas de estas pruebas, y pido disculpas por eso.

Las pruebas que corrí en el archivo terminado fueron las siguientes:

- Intervalo invalido en la búsqueda de rango:



- Intervalo de 27 feb 00:00:00 2025 y la misma fecha y hora pero de 2026:

```
Feb 24 2026 07:33:11 10.14.207.125 Load balancing issues
Feb 24 2026 09:23:28 10.14.172.7 Social engineering attacks
Feb 24 2026 10:09:24 10.14.245.141 Suspicious IP Addresses
Feb 24 2026 12:29:22 10.14.83.26 SQL injection
Feb 24 2026 14:08:17 10.14.146.49 Broken authentication and session management
Feb 24 2026 18:27:46 10.14.61.95 Human error
Feb 24 2026 22:45:08 10.14.17.8 Unauthorized Access Attempts
Feb 25 2026 01:03:17 10.14.89.172 SQL injection
Feb 25 2026 01:36:13 10.14.6.21 Broken authentication and session management
Feb 25 2026 04:56:26 10.14.146.1 Failed Login Attempts
Feb 25 2026 07:04:00 10.14.253.95 Unauthorized Access Attempts
Feb 25 2026 08:05:15 10.14.161.42 SQL injection
Feb 25 2026 11:05:17 10.14.9.59 Data Exfiltration Attempts
Feb 25 2026 22:36:02 10.14.133.47 Failed Login Attempts
Feb 26 2026 02:27:46 10.14.133.24 Human error
Feb 26 2026 07:51:10 10.14.67.157 Weak or easily guessable passwords
Feb 26 2026 09:54:14 10.14.144.236 Weak or easily guessable passwords
Feb 26 2026 11:28:20 10.14.144.67 Encryption Failures
Feb 26 2026 12:48:38 10.14.153.16 Configuration errors
Feb 26 2026 15:34:49 10.14.191.43 Broken authentication and session management
Feb 26 2026 19:08:32 10.14.26.232 Broken authentication and session management

3411 logs encontrados dentro del rango.
Quieres volver a sortear?
1= si, 2= no
1
```

- Shell Sort con algunos datos del output atrás:

```
Logs.cpp U X  main.cpp U X  output608.txt U X  Sorts.h U X  Logs.h U X
Evidences > Evidence1 > out >  output608.txt
1 Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
2 Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
3 Sep 08 2024 04:44:15 10.14.135.12 Application error
4 Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
5 Sep 08 2024 10:42:42 10.14.40.195 Data corruption
6 Sep 08 2024 10:47:52 10.14.24.223 Data corruption
7 Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
8 Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
9 Sep 09 2024 01:51:20 10.14.27.106 Application error
10 Sep 09 2024 01:53:57 10.14.26.200 Suspicious IP addresses
...

Problems Output Debug Console Terminal Ports

Elige uno para ordenar los logs
1. Swap Sort
2. Bubble Sort
3. Selection Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
9. Salir
7
Que tan rapido esperas que sea el sorting con las opciones elegidas?
100

Resultados del sorting:
Algoritmo: Shell_Sort
Database usado: data/log607-2.txt, de tamaño: 6818
Tiempo de ejecucion: 40 ms
Tu prediccion fue: '100'
complejidad: Mejor caso: O(n log n), peor caso: O(n^2)
Nuevo archivo ordenado: out/output608.txt

Busqueda por rango
Ingresa la fecha y hora de inicio. formato ejemplo: Sep 29 2025 10:00:00
|
```

- Valor inválido en menú:

<pre>Quieres volver a sortear? 1= si, 2= no 4 Error, elige 1 o 2 Quieres volver a sortear? 1= si, 2= no </pre>	<pre>Elige una base de datos 1. log607-1.txt (database desordenada) 2. log607-2.txt (database casi ordenada) 5 Selecciona 1 o 2. Elige una base de datos 1. log607-1.txt (database desordenada) 2. log607-2.txt (database casi ordenada) </pre>	<pre>Elige uno para ordenar los logs 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort 9. Salir g Error. Elige del 1 al 9</pre>
---	--	---

- Caso de timestamps iguales:

```
Ingresa la fecha y hora de inicio, formato ejemplo: Sep 29 2025 10:00:00
Nov 09 2025 02:42:01
Ingresa la fecha y hora de fin, formato ejemplo: Sep 29 2025 10:00:00
Dec 16 2025 05:01:57

Registros encontrados:
Nov 09 2025 02:42:01 10.14.86.79 Load balancing issues
Nov 09 2025 02:42:01 10.14.215.213 Failed Login Attempts
Nov 09 2025 04:54:53 10.14.160.129 Load balancing issues
Nov 09 2025 06:05:46 10.14.95.1 SQL injection
Nov 09 2025 06:51:26 10.14.51.161 Human error
Nov 09 2025 07:42:17 10.14.69.33 Port Scanning Activity
Nov 09 2025 08:54:52 10.14.136.68 Anomalous User Activity
Nov 09 2025 10:06:55 10.14.61.91 Cross-Site Scripting
Nov 09 2025 10:18:27 10.14.235.148 Malware Detection Alerts
Nov 09 2025 12:09:10 10.14.14.202 Load balancing issues
Nov 09 2025 19:47:55 10.14.225.96 Social engineering attacks
Nov 09 2025 20:06:50 10.14.219.253 Malware Detection Alerts
Nov 10 2025 03:53:36 10.14.1.38 Encryption Failures
Nov 10 2025 05:11:18 10.14.220.144 Failed Login Attempts
Nov 10 2025 05:54:34 10.14.152.54 Data Exfiltration Attempts
Nov 10 2025 06:09:12 10.14.86.142 SQL injection
Nov 10 2025 07:11:19 10.14.97.251 Encryption Failures
Nov 10 2025 07:24:44 10.14.118.182 Load balancing issues
Nov 10 2025 13:01:59 10.14.211.150 Social engineering attacks
Nov 10 2025 16:36:54 10.14.190.97 Network connectivity issues

Dec 14 2025 21:12:00 10.14.120.00 Human error
Dec 15 2025 01:21:28 10.14.111.209 Cross-Site Scripting
Dec 15 2025 02:31:08 10.14.230.63 Human error
Dec 15 2025 08:35:49 10.14.39.133 Encryption Failures
Dec 15 2025 08:40:40 10.14.196.102 Weak or easily guessable passwords
Dec 15 2025 08:48:03 10.14.21.164 Social engineering attacks
Dec 15 2025 13:53:43 10.14.246.200 Unencrypted communication
Dec 15 2025 16:41:46 10.14.65.124 Malware Detection Alerts
Dec 15 2025 17:46:31 10.14.123.54 Anomalous User Activity
Dec 15 2025 18:36:26 10.14.6.216 Network connectivity issues
Dec 15 2025 19:14:11 10.14.224.1 Human error
Dec 15 2025 19:58:54 10.14.234.166 SQL injection
Dec 15 2025 22:29:05 10.14.46.250 Cross-Site Scripting
Dec 15 2025 23:06:27 10.14.24.69 Anomalous User Activity
Dec 16 2025 04:30:29 10.14.22.152 Application error
Dec 16 2025 05:01:57 10.14.215.155 SQL injection
Dec 16 2025 05:01:57 10.14.246.43 Anomalous User Activity

380 logs encontrados dentro del rango.
Quieres volver a sortear?
1= si, 2= no
```

Tiempo de cada algoritmo

Presenté el cuadro en archivo de Reflexión, pero aquí está nuevamente:

Algoritmo	Desordenados (ms)	Casi ordenados (ms)	Predicción desordenados (ms)	Predicción casi ordenados (ms)
Swap sort	7051	276	1000	2000
Bubble sort	8361	2749	3000	1000
Selection sort	312	261	300	100
Insert sort	6204	14	7000	2000

Merge sort	57	53	1000	30
Quick sort	3802	2789	1000	1000
Shell sort	54	46	300	100

Conclusiones

Conclusión: hallazgos, diferencias observadas entre algoritmos (incluyendo si alguno se comportó de forma muy distinta entre el archivo desordenado y el casi ordenado, y si eso coincide con su complejidad teórica), y recomendaciones.

Algunos algoritmos son notablemente mejores que otros, pero también depende mucho el tipo de dato que se le alimenta. Es decir, para saber qué algoritmo se tiene que ocupar primero se tiene que saber qué dato se va a estar tratando. Pienso que es una buena lección para la programación. Descubrí un poco más de estructura de datos y también de arquitectura de archivos y cómo ocupar la terminal con archivos en diferentes folders. Creo que si pudiera regresar en el tiempo y darme una recomendación sería ocupar nombres de variables más claros y fáciles de entender.